

Prof. Dr. Thomas Wiemann

Robotik Übungsblatt 6

Sommersemester 2026

Aufgabe A6.1 (Gridmapping mit der SLAM Toolbox)

(25 Punkte)

Simultaneous Localization and Mapping (SLAM) beschreibt das Problem, den Roboter zu lokalisieren und gleichzeitig eine konsistente Karte zu erstellen. Zusammengefasst besteht ein auf 2D-Scanmatching basierender SLAM-Algorithmus aus folgenden Schritten:

1. Benutze eine intrinsische Zustandsschätzung des Roboters als a-priori Schätzung (Odometry und IMU mit EKF fusioniert)
2. Benutze die nach dem Scanmatching ermittelte Pose als a-posteriori Filterung zur Korrektur
3. Die beiden ersten Schritte sind anfällig für Drift. Nutze Schleifendetektion und Pose-Graph-Optimierung um diese auszugleichen

Eine gängige Implementierung in ROS2 ist die SLAM-Toolbox¹. Mit diesem Package sind Sie in der Lage, den Roboter anhand der Laserscans und seiner Odometrie zu lokalisieren und dabei gleichzeitig eine Karte zu erstellen. Mithilfe des dort implementierten Algorithmus lässt sich die wahrscheinlichste Pose des Roboters tracken (unimodale Lokalisierung). In dieser Aufgabe machen Sie sich mit der Nutzung dieses Paketes vertraut.

Erstellen Sie mit der SLAM-Toolbox eine 2D-GridMap des Labors und dem angrenzenden Flur. Ein Tutorial dazu finden sie hier². Geben Sie Ihre Karte zusammen mit den Antworten zu folgenden Fragen ab:

- Welche Topics benötigt die SLAM-Toolbox und warum?
- Wie groß kann der zu kartierende Bereich werden, bis der Algorithmus falsche Ergebnisse liefert?
- Was ist eine sinnvolle Kartenauflösung?
- Welche weiteren wichtigen Parameter gibt es und was bewirken sie?

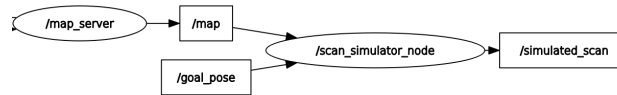
¹https://github.com/SteveMacenski/slam_toolbox

²https://roboticsbackend.com/ros2-nav2-generate-a-map-with-slam_toolbox

Aufgabe A6.2 (Simulation von 2D Laserscans in Rasterkarten)

(25 Punkte)

In dieser Aufgabe werden Sie einen Scan gegeben einer Pose in einer Karte simulieren. Der Nutzer soll eine Pose in der Karte einzeichnen, von dem aus ein LaserScan simuliert werden soll. Der resultierende LaserScan soll danach in RViz im Frame `map` sichtbar sein. Schreiben Sie dazu Node namens `scan_simulator_test` im Package `mcl_helper`. Das in Moodle vorgegebene Programmgerüst stellt die folgenden Publisher und Subscriber zur Verfügung:



Zur Bereitstellung der Karte auf dem Topic `/map` müssen Sie einen `map_server` starten, der die im Verzeichnis `maps` vorgegebene Karte publiziert. Dazu gehen Sie wie folgt vor:

- Starten Sie den Map-Server mit passenden Parametern. Dazu wird Ihnen die Konfigurationsdatei `map_server_params` zur Verfügung gestellt. Starten Sie damit den Map-Server aus dem Stammverzeichnis des Pakets `mcl_helper`:

```
ros2 run nav2_map_server map_server --ros-args --params-file map_server_params.yaml
```

- Der Map-Server ist als sogenannte *Managed Node* implementiert. Diese erlauben es dem ROS2-System ihren Zustand zu melden, um sicherzustellen, dass das System erst dann startet, wenn alle benötigten Nodes in einem gültigen Zustand sind. Entsprechend müssen wir den Map-Server zunächst konfigurieren und aktivieren, bevor wir ihn nutzen können:

```
ros2 lifecycle set /map_server configure
ros2 lifecycle set /map_server activate
```

- Jetzt sollte der Map-Server laufen und auf Anfragen reagieren. Starten Sie nun `RViz2` und fügen Sie einen Visualisierer für das Topic `/map` vom Typ `map` hinzu. Wir wollen alle weiteren Daten im Map-Frame visualisieren, also setzen Sie den `Fixed Frame` in den `Global Options` auf `map`.
- Zuletzt müssen wir sicherstellen, dass die aktuelle Karte auf dem Topic vom Map-Server gepublished wird. Da die Kartendaten potenziell groß sein können, werden dort neue Karten nur veröffentlicht, wenn ein Request gestellt wird. Sollte `RViz2` noch nicht gelaufen sein, wenn Sie den Map-Server gestartet haben, wird das Programm noch keine Daten auf dem `/map`-Topic empfangen haben. Setzen Sie im dem Fall einen Request zum Neuladen der Karte in einem separaten Terminal ab:

```
ros2 service call /map_server/load_map
  nav2_msgs/srv/LoadMap "{map_url: maps/avz_6_floor_gazebo.yaml}"
```

Jetzt sollten Sie die Karten in `RViz2` gerendert sehen.

Übersetzen Sie nun mit `colcon build` das vorgegebene Programmgerüst. Starten Sie die Node `mcl_helper_test` aus dem Paket `mcl_helper`. Diese abonniert das Topic `/map` des Map-Servers und dem Topic `goal_pose`, das von `RViz2` gepubliziert wird. Damit können Sie eine Pose aus `RViz2` heraus senden. Klicken Sie dazu auf den Button *2D Goal Pose*. Dann können Sie einen Pfeil, der eine Position und Orientierung repräsentiert, in der GUI setzen. Diese Daten werden dann als Message vom Typ `geometry_msgs/msg/PoseStamped` versendet. Wenn die Node läuft, sollten Sie dann beim Absetzen einer *2D Goal Pose* von der `mcl_helper_test`-Node die Ausgabe *"Got Pose"* bekommen. Wenn Sie wie oben unter Schritt (d) beschrieben die Karte neu laden, sollte die Ausgabe *"Got Map"* erscheinen. Hinweis: Manchmal wird durch den Request beim ersten Aufruf keine neue Karte auf `/map` veröffentlicht. Setzen Sie in dem Fall den Request mehrfach ab, bis Sie die Bestätigung von der Test-Node erhalten. Wenn Sie beide Nachrichten erzeugen können, haben Sie das System korrekt initialisiert und können jetzt die geforderte Funktionalität implementieren.

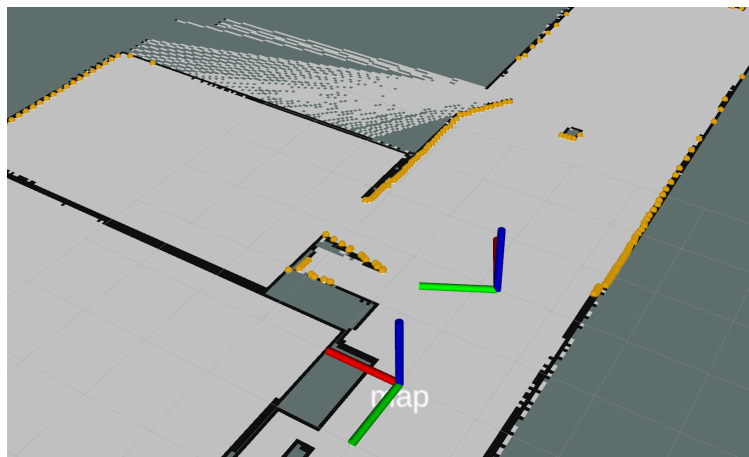
- (a) Fügen Sie der Klasse `mcl_helper_test` eine Instanz der Klasse `mcl_helper::ScanSimulator` hinzu. Mit Hilfe dieser Klasse können Sie Laserscans in 2D-Gridmaps an gegebenen Posen simulieren.
- (b) Sobald Sie eine neue Karte erhalten, fügen Sie diese mittels `setMap` in den Simulator ein.
- (c) Sobald Sie eine Pose auf dem Topic `goal_pose` bekommen, simulieren Sie einen Scan mittels der Methode `simulateScan` des Scan-Simulators. Diese schreibt die simulierten Daten automatisch in die Daten-Felder der übergebenen `sensor_msgs::msg::LaserScan`. Damit der Simulator weiß, mit welchen Sensor-Parametern er die Ergebnisse berechnen soll, müssen Sie zunächst die Scanner-Parameter in der `sensor_msgs::msg::LaserScan` Struktur setzen. Diese werden bei der Simulation zunächst ausgelesen, bevor die synthetischen Scan-Daten berechnet werden. Geeignete Werte sind:

```
angle_min = -1.5708
angle_max = +1.5708
range_min = 0;
range_max = 10
angle_increment = 0.0174553
```

Nach der Simulation veröffentlichen Sie die Nachricht im Frame `scan_simulator` auf dem Topic `/simulated_scan`.

- (d) Die simulierten Daten werden im Frame `scan_simulator` berechnet. Damit sie korrekt in der Karte angezeigt werden, müssen wir noch die Transformationen zwischen `/map` und `/scan_simulator` bekannt geben. Dazu nutzen wir einen `tf2_ros::TransformBroadcaster` in der Klasse `mcl_helper_test`. Erzeugen sie Dazu eine Struktur des Typs `geometry_msgs::msg::TransformStamped` mit aktuellen Zeitstempel (Methode `now`) und `/map` als Frame und `/scan_simulator` als Child-ID. Füllen Sie die Transformation mit den Daten aus der empfangenen `geometry_msgs::msg::PoseStamped`.

Wenn Sie alles korrekt implementiert haben, sollten Sie bei einer Visualisierung des Topic `/simulated_scan` eine korrekte Korrespondenz mit der Karte sehen. Dies könnte z.B. so aussehen:



Aufgabe B6.1 (Markow Lokalisierung)

(50 Punkte)

Gegeben sei die in Abbildung ?? abgebildete Welt eines mobilen Roboters. Der Roboter bewege sich deterministisch in einem Korridor mit 16 Gitterzellen. In einigen dieser Zellen sind Landmarken angebracht. Wenn der Roboter sich in einer der Zellen mit Landmarken befindet, so wird diese Landmarke von dem Roboter mit einer Wahrscheinlichkeit von 70% detektiert. In allen anderen Zellen (ohne Landmarken) meldet der Roboter aufgrund der Sensorungenauigkeit mit 25% Wahrscheinlichkeit dennoch eine Landmarke.

Der Roboter führt die folgenden 5 Aktionen aus:

1. Der Roboter detektiert eine Landmarke.
2. Der Roboter bewegt sich 2 Zellen im Uhrzeigersinn.
3. Der Roboter detektiert wieder eine Landmarke.
4. Der Roboter bewegt sich 4 Zellen im Uhrzeigersinn.
5. Der Roboter detektiert keine Landmarke.

Berechnen Sie die Aufenthaltswahrscheinlichkeit für den Roboter für jede Zelle nach jedem einzelnen Schritt. In welcher Zelle ist demnach der Roboter vermutlich gestartet?

	1	2	3		4	5
16						6
15						7
14						8
	13	12	11	10		9

Nehmen Sie nun an, dass bei einer Bewegung des Roboters zwar bekannt ist, wie weit sich der Roboter bewegt hat, nicht aber in welche Richtung (im oder entgegen dem Uhrzeigersinn). Berechnen Sie auch hier die Aufenthaltswahrscheinlichkeiten.